

machine, he then just needs to breach the hypervisor running on the host. If there is a hypervisor vulnerability currently existing in the wild, he can exploit it and can thus, eventually, take over the rest of the virtual machines running on the host.

Understanding sVirt

sVirt was introduced as a feature of Fedora 11, and conceived as a security mechanism for Linux-based virtualisation schemes like the (KVM)/Qemu, and Iguest. It has been developed primarily by Dan Walsh and James Morris, of Red Hat. Some of the goals/use-cases of sVirt are listed below:

- Providing virtualised services with a level of isolation similar to that previously afforded by physical separation.
- Increased protection of the host from untrustworthy VM guests (for example, for VM hosting providers, grid/cloud servers, etc).
- Increased protection of VM guests from each other in the event of host flaws or misconfiguration. Some protection may also be provided against a compromised host.
- Strongly isolating desktop applications by running them in separately labelled VMs (for example, online banking in one VM and *World of Warcraft* in another; opening untrustworthy office documents in an isolated VM for view/print only).

How does it work?

sVirt essentially works by adding SELinux support to the Linux virtualisation library libvirt. It is designed as a pluggable security framework into the libvirt API (Figure 3). It also supports other MAC security schemes like SMACK (Simplified Mandatory Access Control Kernel), a security kernel module. In a typical use, one ought not to even notice sVirt running in the background.

As you already know, SELinux works by labelling everything like files, devices, processes, etc. The libvirt daemon (*libvirt*) starts all virtual machines. And all virtual machines run as separate processes. libvirt dynamically generates labels for the image files and starts the virtual machines with the appropriate labels. The SELinux policy has rules that allow the *svirt_t* processes to read/write *svirt_image_t* files and devices.

This allows us to protect the host machine from any of its VM guests. A virtual machine will only be able to interact with the files and devices with the correct labels. A compromised virtual machine would not be allowed to read my home directory, for example, even if the virtual machine is running as the root.

However, this 'type' protection does not prevent one virtual machine from attacking another. We need a way to label the domains and the image files with the same TYPES. Yet, at the same time, we need to stop virtual machine 1 (running as svirt_t) from attacking virtual machine 2, which would also be running as svirt_t.

This is where the Multi Category Security (MCS) mechanism comes into play. MCS is an enhancement

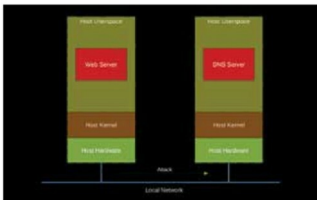


Figure 1: Server set-ups in the absence of virtualisation

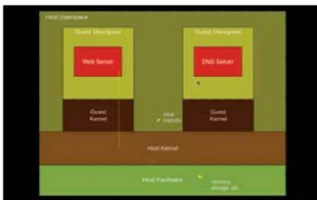


Figure 2: Server set-ups using virtualisation

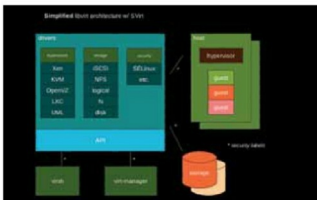


Figure 3: Simplified libvirt architecture with sVirt integration

to SELinux, which allows users to label files with categories. These categories are simply text labels, such as 'Company_Confidential' or 'Medical_Records'. The sysadmin first configures the categories, then assigns users to them, as required. In a nutshell, to access a file, a user needs to be assigned to all of the categories with which the file is labelled.

Note: The first thing the sysadmin will want to do with MCS is configure some categories. This can be done by editing the `/etc/selinux/targeted/setrans.conf` file. This file specifies how the internal MCS categories can be mapped to human-